

KERNEL AUTHORIZING WITH CUDA PYTHON

Bryce Adelstein Lelbach

Principal Architect

✉ brycelelbach@gmail.com [@blelbach](https://twitter.com/blelbach)



Grid-Level

Easy acceleration of
bulk data operations



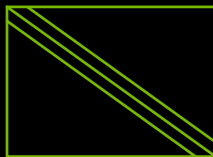
System splits work
into blocks, divides
data into tiles, & maps
both onto threads

```
import cupy as cp

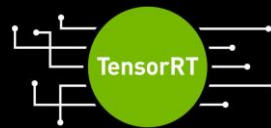
samples = ...
sort     = cp.sort(samples)
cdf      = cp.cumsum(sort) / sort.sum()
p95      = sort[cp.searchsorted(cdf, 0.95)]
```



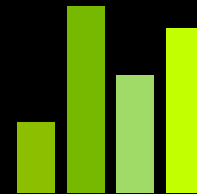
cuBLAS



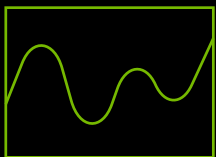
cuSPARSE



**cuFile
(GDS)**



NVBench



cuFFT

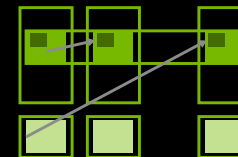


cuDSS

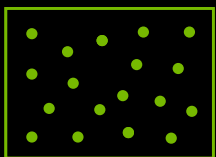
**CV
CUDA**

cuLitho

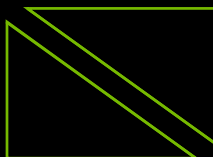
**Cooperative
Groups**



NVSHMEM



cuRAND

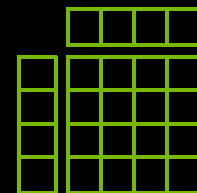


cuSOLVER

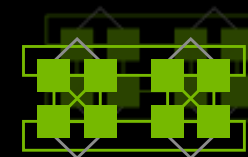
RAPIDS



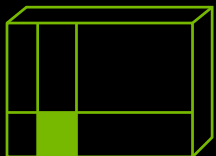
CUDA-Q



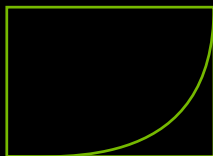
CUTLASS



NCCL



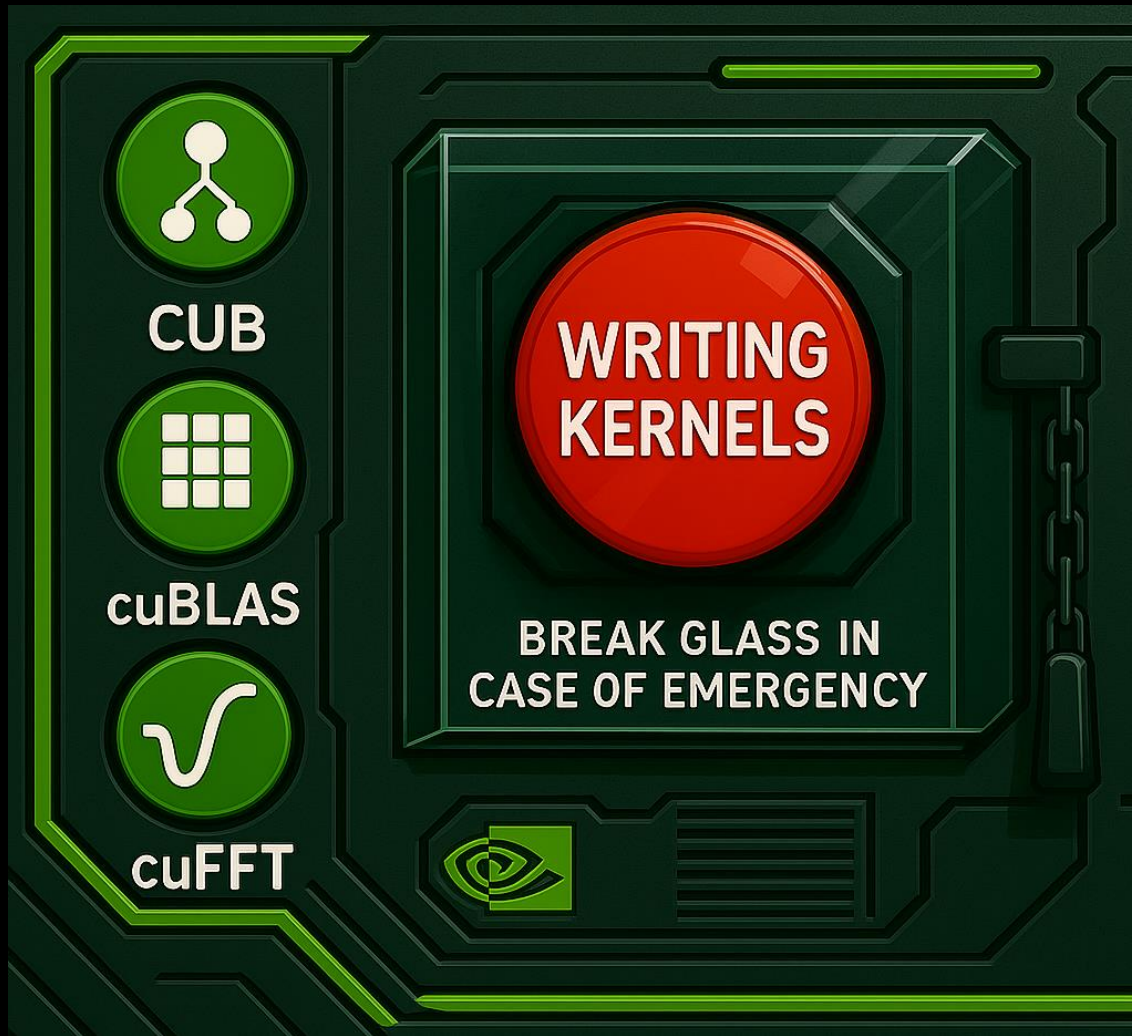
cuTENSOR



CUDA Math



**CUDA
Runtime**



Prefer using libraries!

If no library exists or
you can't get the perf
you need with them,
then...

```

@cuda.device.kernel
def matmul(A, B, C):
    TPB = cuda.blockDim.x
    BPG = cuda.gridDim.x
    x, y = cuda.grid(2)
    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y

    sA = cuda.shared.array((TPB, TPB), float32)
    sB = cuda.shared.array((TPB, TPB), float32)

    tmp = float32(0.)
    for i in range(BPG):
        sA[ty, tx] = 0
        sB[ty, tx] = 0
        if y < A.shape[0] and (tx+i*TPB) < A.shape[1]:
            sA[ty, tx] = A[y, tx + i * TPB]
        if x < B.shape[1] and (ty+i*TPB) < B.shape[0]:
            sB[ty, tx] = B[ty + i * TPB, x]

        cuda.syncthreads()
        for j in range(TPB):
            tmp += sA[ty, j] * sB[j, tx]
        cuda.syncthreads()

    if y < C.shape[0] and x < C.shape[1]:
        C[y, x] = tmp

```

```

@cuda.device.kernel
def matmul(A, B, C):
    TPB = cuda.blockDim.x
    BPG = cuda.gridDim.x
    x, y = cuda.grid(2)
    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y

    sA = cuda.shared.array((TPB, TPB), float32)
    sB = cuda.shared.array((TPB, TPB), float32)

    tmp = float32(0.)
    for i in range(BPG):
        sA[ty, tx] = 0
        sB[ty, tx] = 0
        if y < A.shape[0] and (tx+i*TPB) < A.shape[1]:
            sA[ty, tx] = A[y, tx + i * TPB]
        if x < B.shape[1] and (ty+i*TPB) < B.shape[0]:
            sB[ty, tx] = B[ty + i * TPB, x]

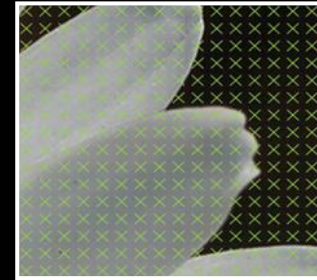
        cuda.syncthreads()
        for j in range(TPB):
            tmp += sA[ty, j] * sB[j, tx]
        cuda.syncthreads()

    if y < C.shape[0] and x < C.shape[1]:
        C[y, x] = tmp

```

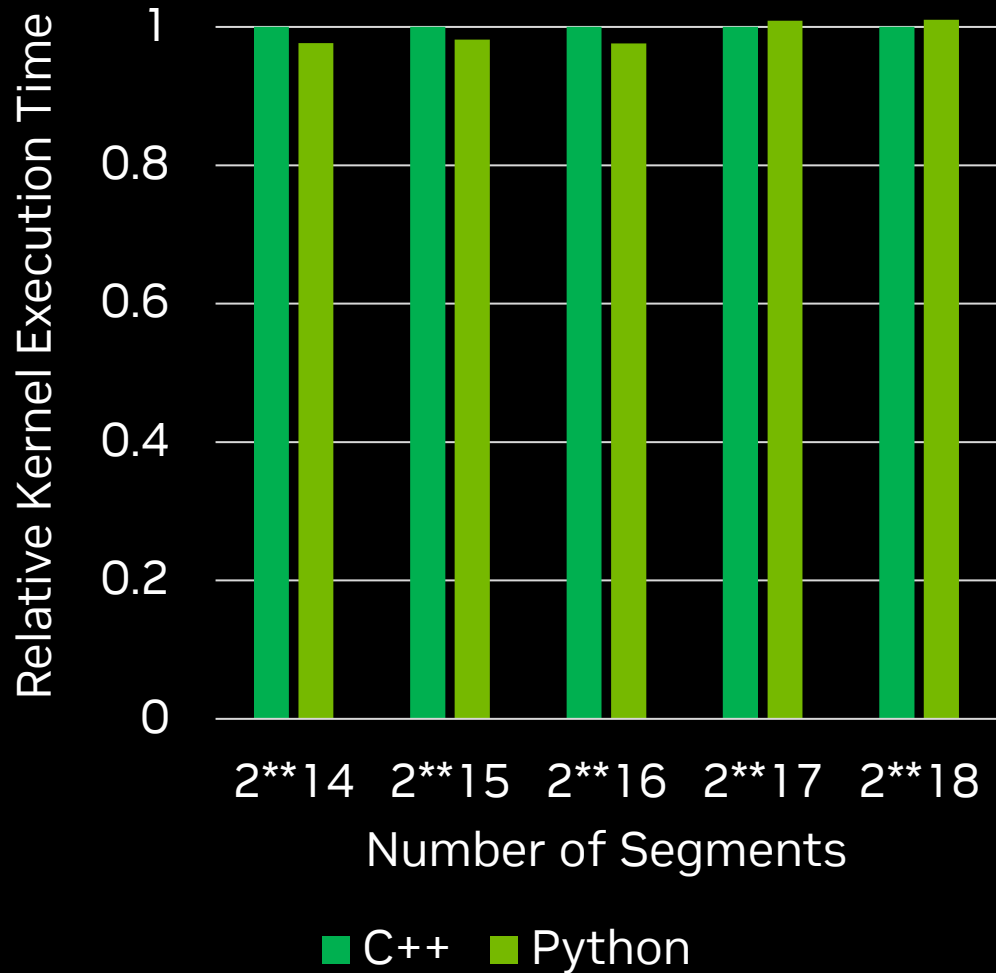
Thread-Level

Full control for
all parallel workloads

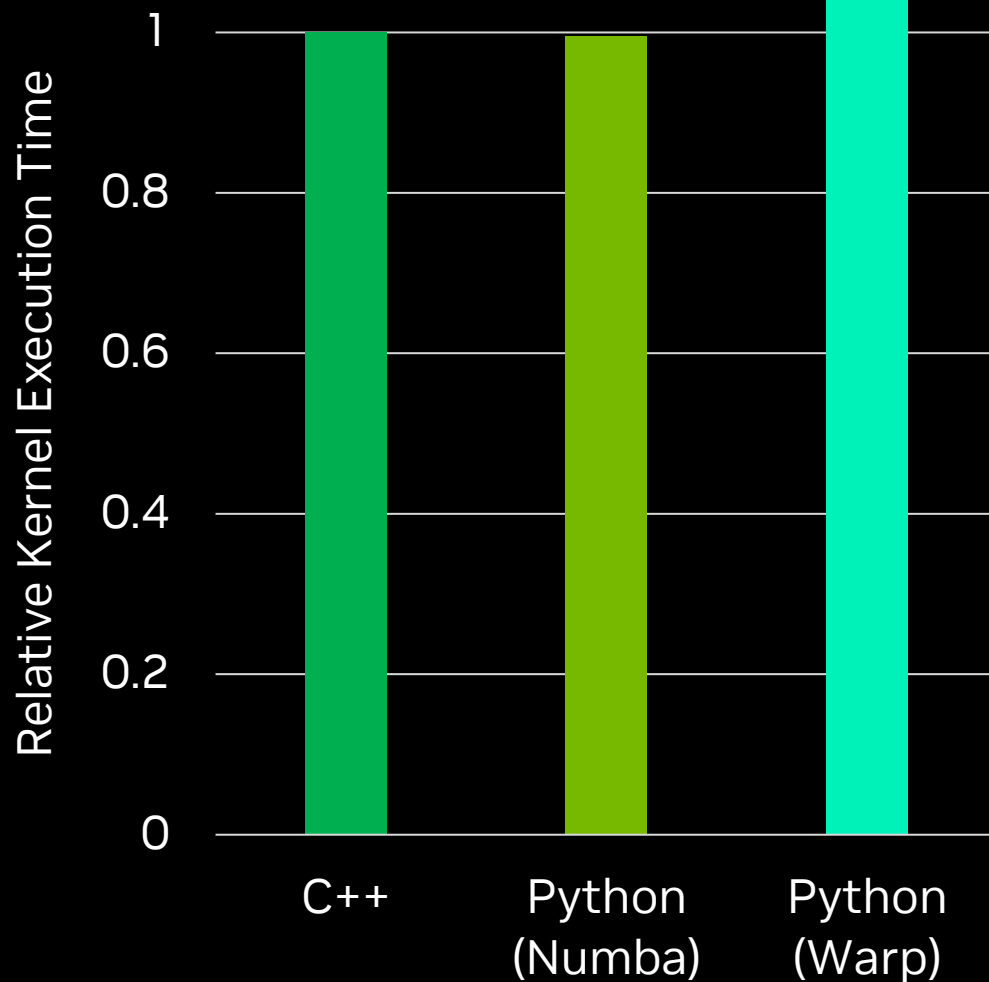


User splits work into
blocks, divides data into
tiles, & maps both onto
threads

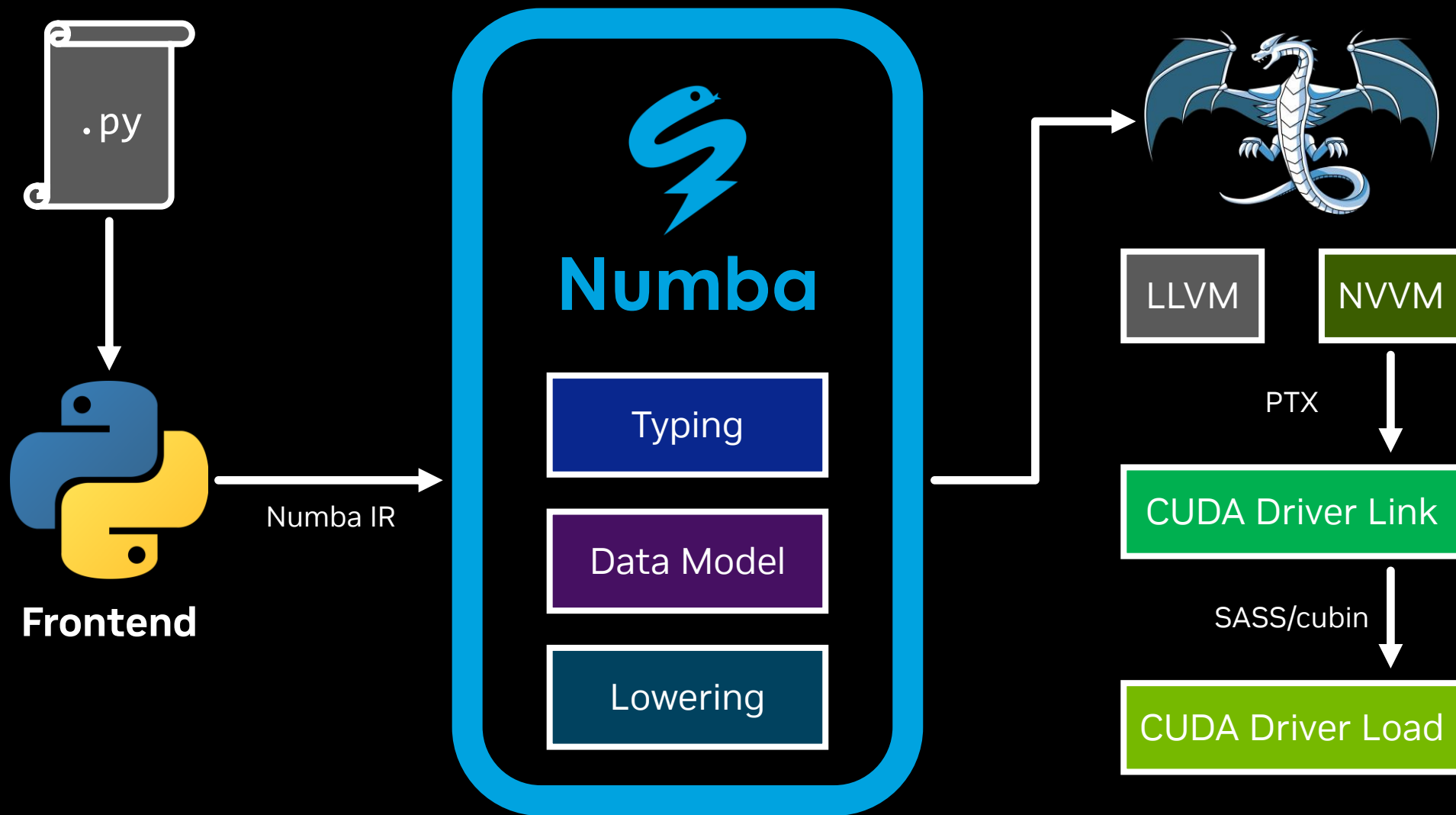
CUDA Segmented Sort Kernel

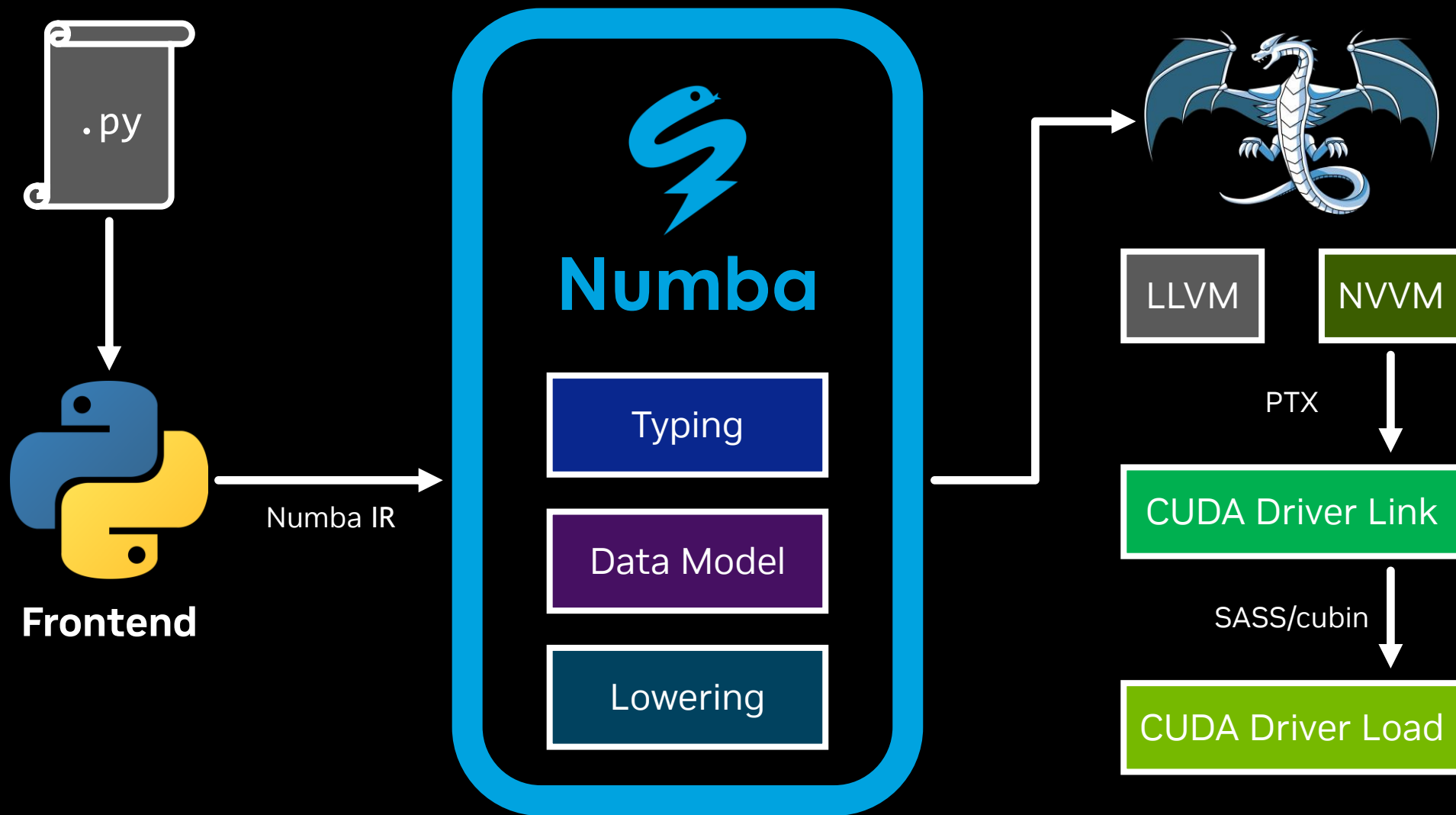


CUDA FFT Convolution Kernel



A6000, CUDA 12.6, ["CUDA: New Features and Beyond", GTC 2025](#)





```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

```
from numba import cuda
```

```
@cuda.jit
```

```
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```



```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

$d[0]=s[0]$



$d[1]=s[1]$



$d[2]=s[2]$



$d[3]=s[3]$



$d[4]=s[4]$



$d[5]=s[5]$



$d[6]=s[6]$

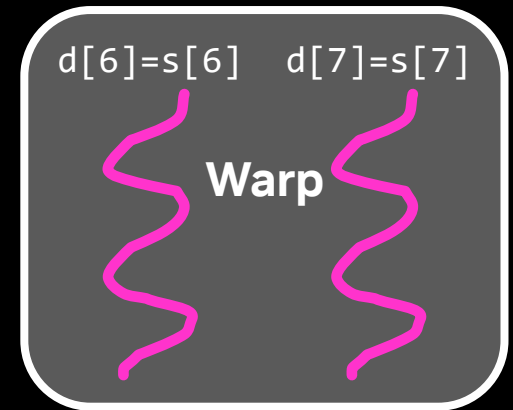
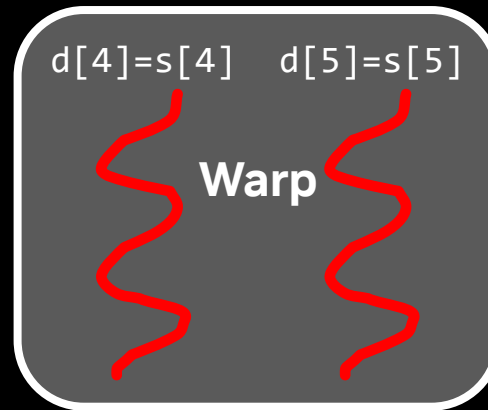
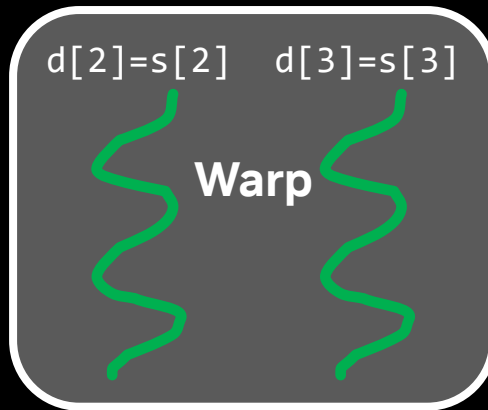
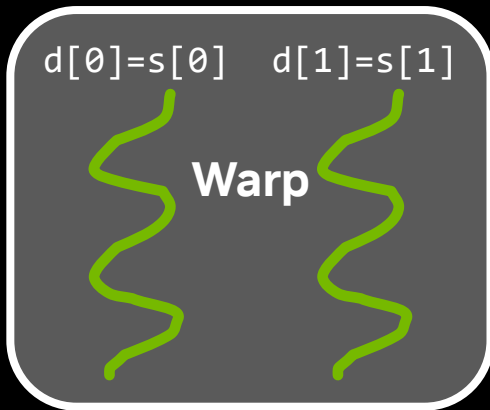


$d[7]=s[7]$



```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```



```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

Block 0

d[0]=s[0] d[1]=s[1]



Warp
0

d[2]=s[2] d[3]=s[3]



Warp
1

Block 1

d[4]=s[4] d[5]=s[5]



Warp
0

d[6]=s[6] d[7]=s[7]



Warp
1

```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

```
s = cp.arange(N)  
d = cp.empty_like(s)  
copy[2, 4](s, d)
```

Block 0

d[0]=s[0] d[1]=s[1]



d[2]=s[2] d[3]=s[3]



Block 1

d[4]=s[4] d[5]=s[5]



d[6]=s[6] d[7]=s[7]



```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

```
s = cp.arange(N)  
d = cp.empty_like(s)  
copy[2, 4](s, d)
```

Block 0

d[0]=s[0] d[1]=s[1]



d[2]=s[2] d[3]=s[3]



Block 1

d[4]=s[4] d[5]=s[5]



d[6]=s[6] d[7]=s[7]



```
from numba import cuda
```

```
@cuda.jit  
def copy(s, d):  
    t = cuda.grid(1)  
    d[t] = s[t]
```

```
s = cp.arange(N)  
d = cp.empty_like(s)  
copy[2, 4](s, d)
```

Block 0

d[0]=s[0] d[1]=s[1]



d[2]=s[2] d[3]=s[3]



Block 1

d[4]=s[4] d[5]=s[5]



d[6]=s[6] d[7]=s[7]



```
@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[...](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[...](s, d, T)
```



```
@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[...](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]
```

```
s = cp.arange(N)
d = cp.empty_like(s)
copy[...](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T))](s, d, T)
```

N = Size of the arrays

B = Number of thread blocks

T = Items per thread

```
@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

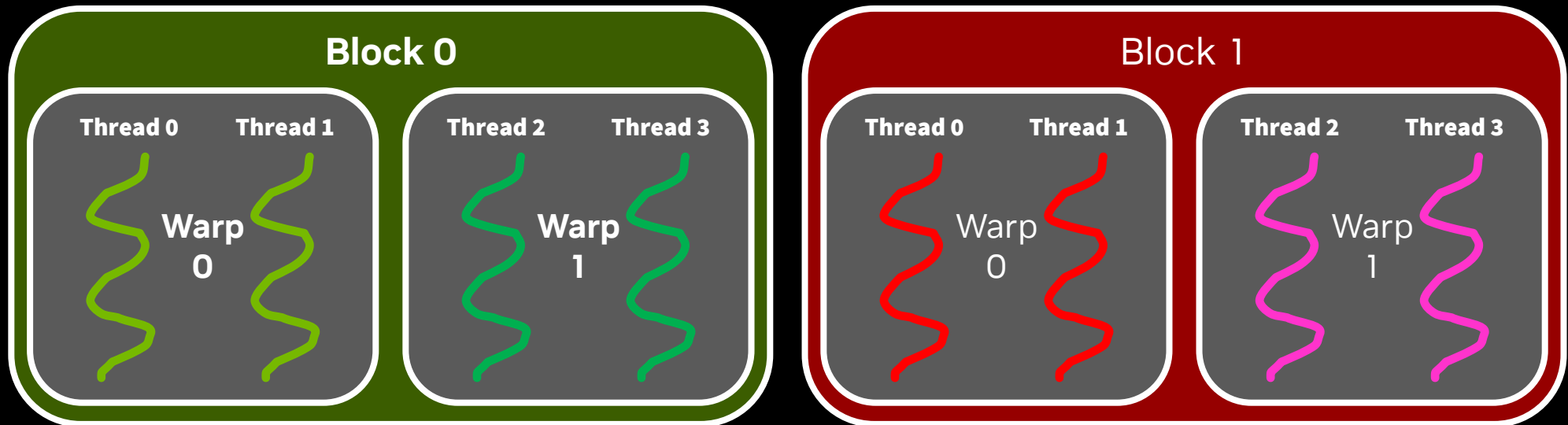
s = cp.arange(N)
d = cp.empty_like(s)
copy(int(N/(B*T)), B](s, d, T)
```

N = Size of the arrays

B = Number of thread blocks

T = Items per thread

`cuda.grid(1)` = Global thread index (unique)
`cuda.threadIdx.x` = Block-local thread index
`cuda.threadDim.x` = # of threads per block
`cuda.blockIdx.x` = Global block index (unique)
`cuda.blockDim.x` = # of blocks



```

@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

```

Step	Thread 0	Thread 1	Thread 2	Thread 3
load	d[0]	d[4]	d[8]	d[12]
store	s[0]	s[4]	s[8]	s[12]
load	d[1]	d[5]	d[9]	d[13]
store	s[1]	s[5]	s[9]	s[13]
load	d[2]	d[6]	d[10]	d[14]
store	s[2]	s[6]	s[10]	s[14]
load	d[3]	d[7]	d[11]	d[15]
store	s[3]	s[7]	s[11]	d[15]

...

...

...

...

...

```

@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

```

Step	Thread 0	Thread 1	Thread 2	Thread 3
load	d[0]	d[4]	d[8]	d[12]
store	s[0]	s[4]	s[8]	s[12]
load	d[1]	d[5]	d[9]	d[13]
store	s[1]	s[5]	s[9]	s[13]
load	d[2]	d[6]	d[10]	d[14]
store	s[2]	s[6]	s[10]	s[14]
load	d[3]	d[7]	d[11]	d[15]
store	s[3]	s[7]	s[11]	d[15]

...

...

...

...

...

```

@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

```

Step	Thread 0	Thread 1	Thread 2	Thread 3
load	d[0]	d[4]	d[8]	d[12]
load	d[1]	d[5]	d[9]	d[13]
load	d[2]	d[6]	d[10]	d[14]
load	d[3]	d[7]	d[11]	d[15]
store	s[0]	s[4]	s[8]	s[12]
store	s[1]	s[5]	s[9]	s[13]
store	s[2]	s[6]	s[10]	s[14]
store	s[3]	s[7]	s[11]	d[15]

...

...

...

...

...


```

@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

```

Step	Thread 0	Thread 1	Thread 2	Thread 3
load	d[0:3]	d[4:7]	d[8:11]	d[12:15]
store	s[0:3]	s[4:7]	s[8:11]	s[12:15]
...	...	...	...	...

```

@cuda.jit
def copy(s, d, items):
    t = cuda.grid(1) * items
    for i in range(items):
        d[t + i] = s[t + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

```

Step	Thread 0	Thread 1	Thread 2	Thread 3
load	d[0]	d[4]	d[8]	d[12]
store	s[0]	s[4]	s[8]	s[12]
load	d[1]	d[5]	d[9]	d[13]
store	s[1]	s[5]	s[9]	s[13]
load	d[2]	d[6]	d[10]	d[14]
store	s[2]	s[6]	s[10]	s[14]
load	d[3]	d[7]	d[11]	d[15]
store	s[3]	s[7]	s[11]	d[15]

...

...

...

...

...

```
@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)
```



```
@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)
```

```
@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)
```

```

@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

```

Step	Thread 0	Thread 1	Thread 2	Thread 3
load	d[0]	d[1]	d[2]	d[3]
store	s[0]	s[1]	s[2]	s[3]
load	d[4]	d[5]	d[6]	d[7]
store	s[4]	s[5]	s[6]	s[7]
load	d[8]	d[9]	d[10]	d[11]
store	s[8]	s[9]	s[10]	s[11]
load	d[12]	d[13]	d[14]	d[15]
store	s[12]	s[13]	s[14]	d[15]
...	...	...	...	...

```

@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

```

Step	Thread [0:3]
load	d[0:3]
store	s[0:3]
load	d[4:7]
store	s[4:7]
load	d[8:11]
store	s[8:11]
load	d[12:15]
store	s[12:15]

... ..

```

@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

```

Step	Thread [0:3]
load	d[0:3]
load	d[4:7]
load	d[8:11]
load	d[12:15]
store	s[0:3]
store	s[4:7]
store	s[8:11]
store	s[12:15]

... ...

```

@cuda.jit
def copy(s, d, items):
    bd = cuda.blockDim.x
    bx = cuda.blockIdx.x
    tx = cuda.threadIdx.x
    tile = bd * items

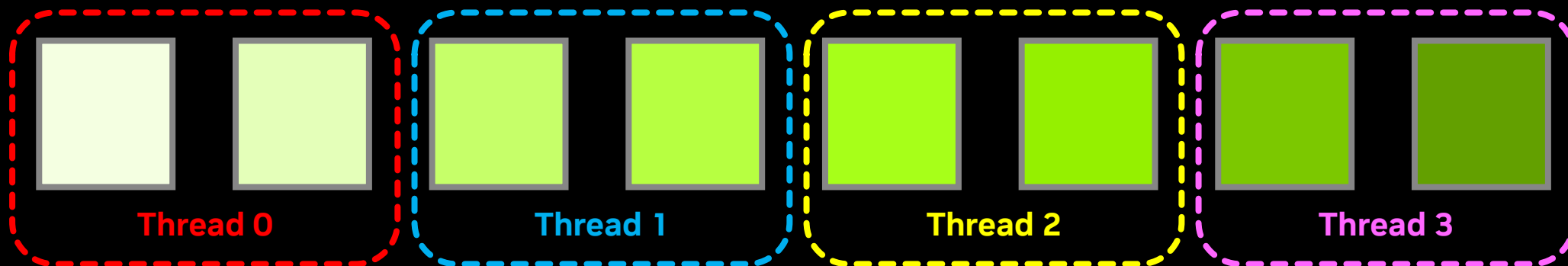
    base = tx + bx * tile
    for i in range(0, tile, bd):
        d[base + i] = s[base + i]

s = cp.arange(N)
d = cp.empty_like(s)
copy[int(N/(B*T)), B](s, d, T)

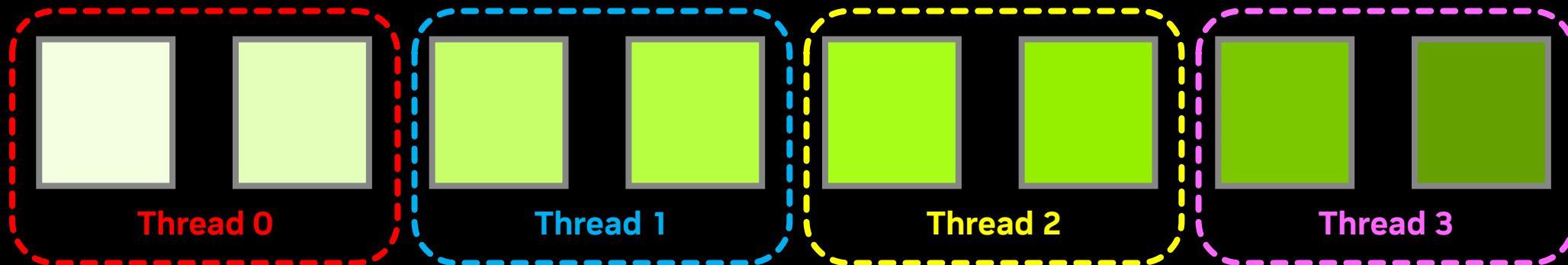
```

Step	Thread [0:3]
load	d[0:15]
store	s[0:15]
...	...

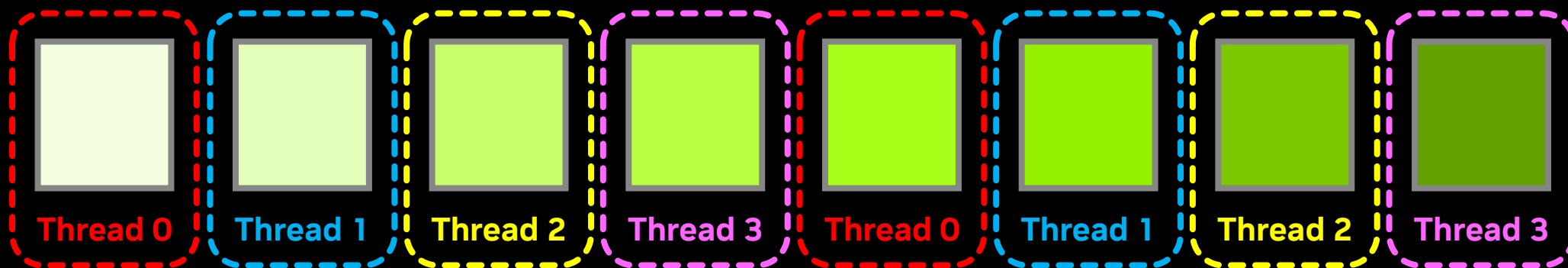
Blocked Arrangement



Blocked Arrangement



Striped Arrangement





Cooperative primitives for CUDA C++

<https://nvidia.github.io/cccl/cub>

```
template <int BLOCK_THREADS, int ITEMS>
__global__ void BlockSortKernel(int *d_in, int *d_out)
{
    using BlockLoadT = cub::BlockLoad<
        int, BLOCK_THREADS, ITEMS, cub::BLOCK_LOAD_TRANSPOSE>;
    using BlockStoreT = cub::BlockStore<
        int, BLOCK_THREADS, ITEMS, cub::BLOCK_STORE_TRANSPOSE>;
    using BlockRadixSortT = cub::BlockRadixSort<
        int, BLOCK_THREADS, ITEMS>;

    // Allocate type-safe, repurposable shared memory
    __shared__ union {
        typename BlockLoadT::TempStorage    load;
        typename BlockStoreT::TempStorage    store;
        typename BlockRadixSortT::TempStorage sort;
    } temp;

    // Obtain this block's segment of consecutive keys
    int thread_keys[ITEMS];
    int offset = blockIdx.x * (BLOCK_THREADS * ITEMS);
    BlockLoadT(temp.load).Load(d_in + offset, thread_keys);

    __syncthreads();           // Barrier for smem reuse

    // Collectively sort the keys
    BlockRadixSortT(temp.sort).Sort(thread_keys);

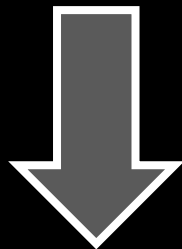
    __syncthreads();           // Barrier for smem reuse

    // Store the sorted segment
    BlockStoreT(temp.store).Store(d_out + offset, thread_keys);
}
```



Cooperative primitives for CUDA C++

<https://nvidia.github.io/cccl/cub>



```
import cuda.coop
```

```
template <int BLOCK_THREADS, int ITEMS>
__global__ void BlockSortKernel(int *d_in, int *d_out)
{
    using BlockLoadT = cub::BlockLoad<
        int, BLOCK_THREADS, ITEMS, cub::BLOCK_LOAD_TRANSPOSE>;
    using BlockStoreT = cub::BlockStore<
        int, BLOCK_THREADS, ITEMS, cub::BLOCK_STORE_TRANSPOSE>;
    using BlockRadixSortT = cub::BlockRadixSort<
        int, BLOCK_THREADS, ITEMS>;

    // Allocate type-safe, repurposable shared memory
    __shared__ union {
        typename BlockLoadT::TempStorage    load;
        typename BlockStoreT::TempStorage    store;
        typename BlockRadixSortT::TempStorage sort;
    } temp;

    // Obtain this block's segment of consecutive keys
    int thread_keys[ITEMS];
    int offset = blockIdx.x * (BLOCK_THREADS * ITEMS);
    BlockLoadT(temp.load).Load(d_in + offset, thread_keys);

    __syncthreads();           // Barrier for smem reuse

    // Collectively sort the keys
    BlockRadixSortT(temp.sort).Sort(thread_keys);

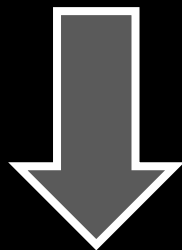
    __syncthreads();           // Barrier for smem reuse

    // Store the sorted segment
    BlockStoreT(temp.store).Store(d_out + offset, thread_keys);
}
```



Cooperative primitives for CUDA C++

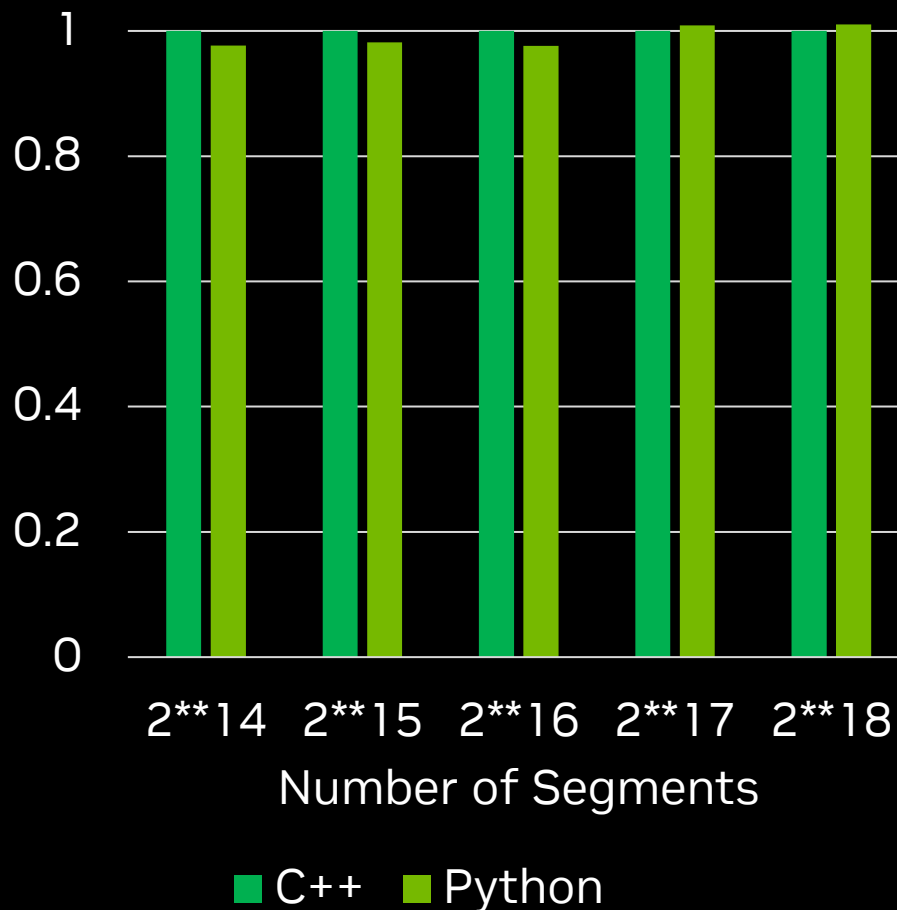
<https://nvidia.github.io/cccl/cub>



```
import cuda.coop
```

CUDA Segmented Sort Kernel

Relative Kernel Execution Time



A6000, CUDA 12.6, ["CUDA: New Features and Beyond", GTC 2025](#)

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop
```

```
N = 2**28 # Size of the input/output
```

```
B = 256    # Number of thread in each block
```

```
T = 64     # Number of items in each thread's registers
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')
```



```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def copy(src, dst):
    ...
```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def copy(src, dst):
    locals = cuda.local.array(T, dtype=cp.int32)
```

```

import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def copy(src, dst):
    locals = cuda.local.array(T, dtype=cp.int32)
    base = cuda.blockIdx.x * B * T

```

```
import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def copy(src, dst):
    locals = cuda.local.array(T, dtype=cp.int32)
    base = cuda.blockIdx.x * B * T

    block_load(src[base : base + B * T], locals)
```



```

import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def copy(src, dst):
    locals = cuda.local.array(T, dtype=cp.int32)
    base = cuda.blockIdx.x * B * T

    block_load(src[base : base + B * T], locals)
    block_store(dst[base : base + B * T], locals)

```

```

import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def copy(src, dst):
    locals = cuda.local.array(T, dtype=cp.int32)
    base = cuda.blockIdx.x * B * T

    block_load(src[base : base + B * T], locals)
    block_store(dst[base : base + B * T], locals)

src = np.arange(N, dtype='int32')
dst = np.empty_like(src)
copy[int(N / (B * T)), B](src, dst)
assert np.all(src == dst)

```

```

import cupy as cp
import numba.cuda as cuda
import cuda.coop as coop

N = 2**28 # Size of the input/output
B = 256   # Number of thread in each block
T = 64    # Number of items in each thread's registers

block_load  = coop.block.load(cp.int32, B, T, 'striped')
block_store = coop.block.store(cp.int32, B, T, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def copy(src, dst):
    locals = cuda.local.array(T, dtype=cp.int32)
    base = cuda.blockIdx.x * B * T

    block_load(src[base : base + B * T], locals)
    block_store(dst[base : base + B * T], locals)

src = np.arange(N, dtype='int32')
dst = np.empty_like(src)
copy[int(N / (B * T)), B](src, dst)
assert np.all(src == dst)

```

"Whose woods these are I think I know."

"Whose woods these are I think I know."



"Whose woods these are I think I know."

h
1

w
1

"Whose woods these are I think I know."

h
1

o
1

w
1

"Whose woods these are I think I know."

h
1

o
1

s
1

w
1

"Whose woods these are I think I know."

e	h
1	1

o
1

s
1

w
1

"Whose woods these are I think I know."

e	h
1	1

0
1

S
1

W
1



"Whose woods these are I think I know."

e	h		o	s	w	
1	1		1	1	2	1

"Whose woods these are I think I know."

e	h		o	s	w	
1	1		2	1	2	1

"Whose woods these are I think I know."

e	h		o	s	w	
1	1		3	1	2	1

"Whose woods these are I think I know."

d	e	h
1	1	1

o
3

s
1

w
2

1

"Whose woods these are I think I know."

d	e	h
1	1	1

o
3

s
2

w
2

1

"Whose woods these are I think I know."

a	d	e	h	i	k	n	o	r	s	t	w	.	
1	1	4	3	3	2	2	4	1	3	2	3	1	7


```
@cuda.jit
def histogram(values, bins, items):
    for i in range(items):
        value = values[cuda.grid(1) * items + i]
        old_count = bins[value]
        new_count = old_count + 1
        bins[value] = new_count
```

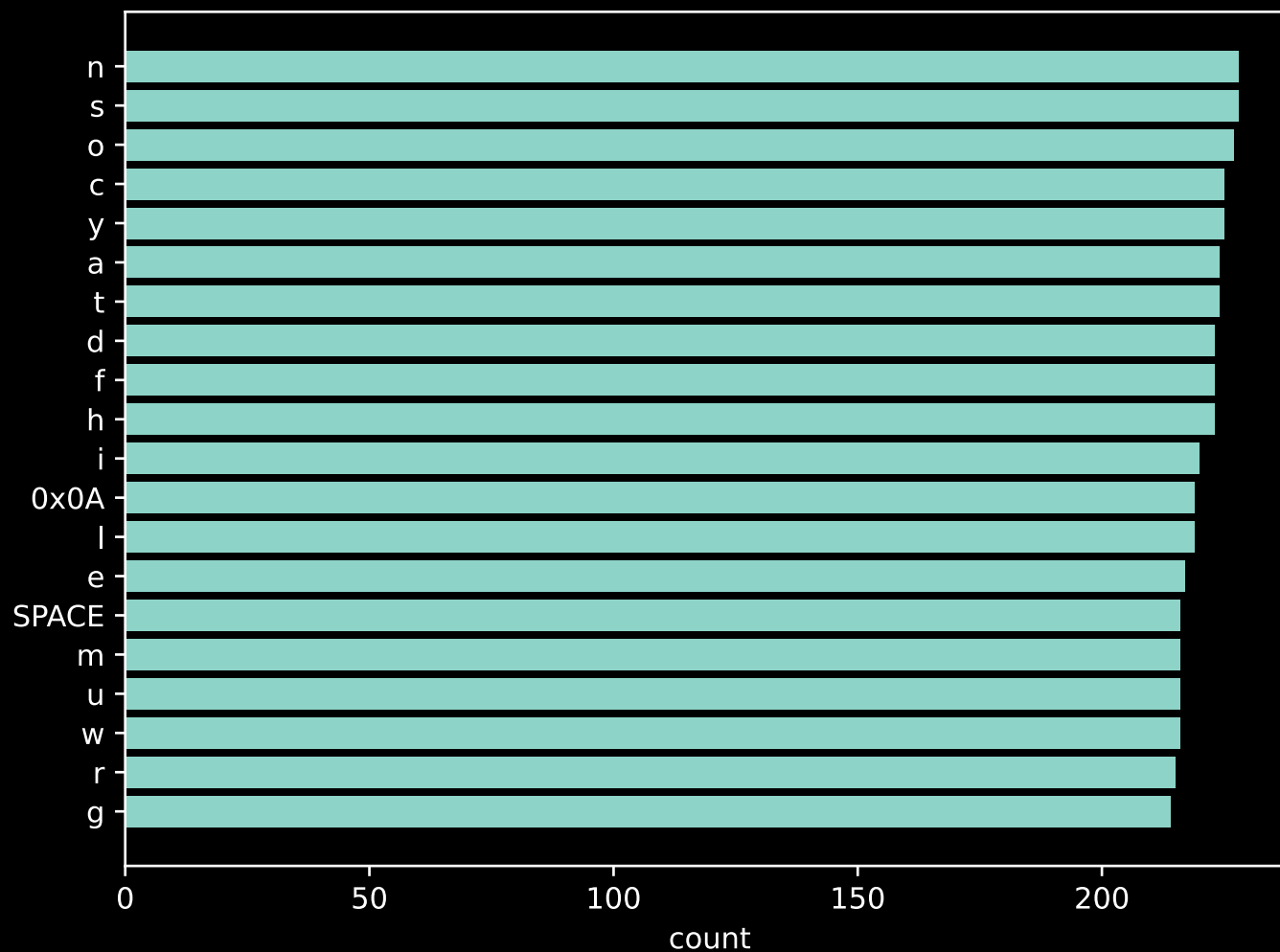
```
@cuda.jit
def histogram(values, bins, items):
    for i in range(items):
        value = values[cuda.grid(1) * items + i]
        old_count = bins[value]
        new_count = old_count + 1
        bins[value] = new_count
```

```
@cuda.jit
def histogram(values, bins, items):
    for i in range(items):
        value = values[cuda.grid(1) * items + i]
        old_count = bins[value]
        new_count = old_count + 1
        bins[value] = new_count
```

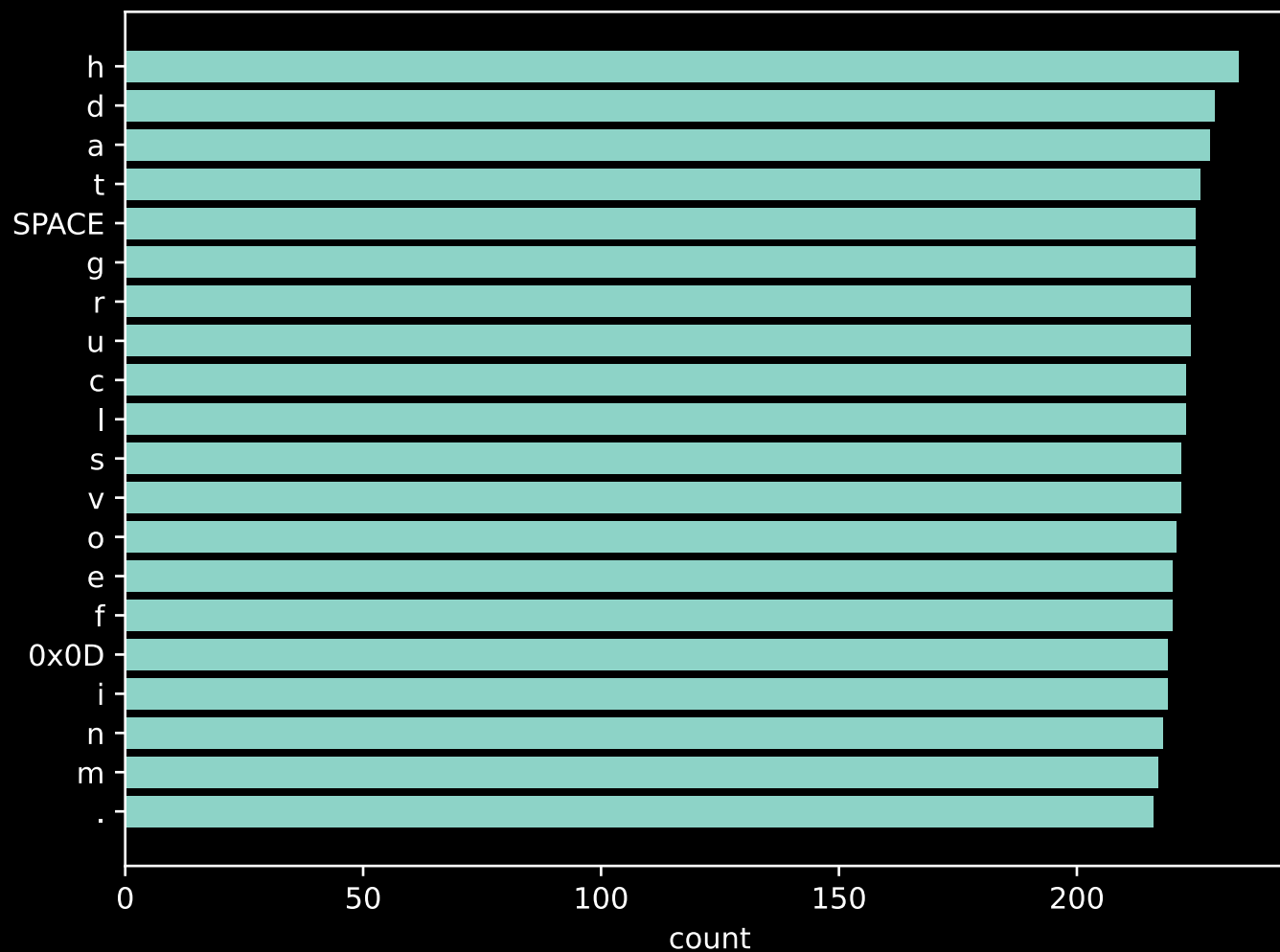
```
@cuda.jit
def histogram(values, bins, items):
    for i in range(items):
        value = values[cuda.grid(1) * items + i]
        old_count = bins[value]
        new_count = old_count + 1
        bins[value] = new_count
```

```
@cuda.jit
def histogram(values, bins, items):
    for i in range(items):
        value = values[cuda.grid(1) * items + i]
        old_count = bins[value]
        new_count = old_count + 1
        bins[value] = new_count
```

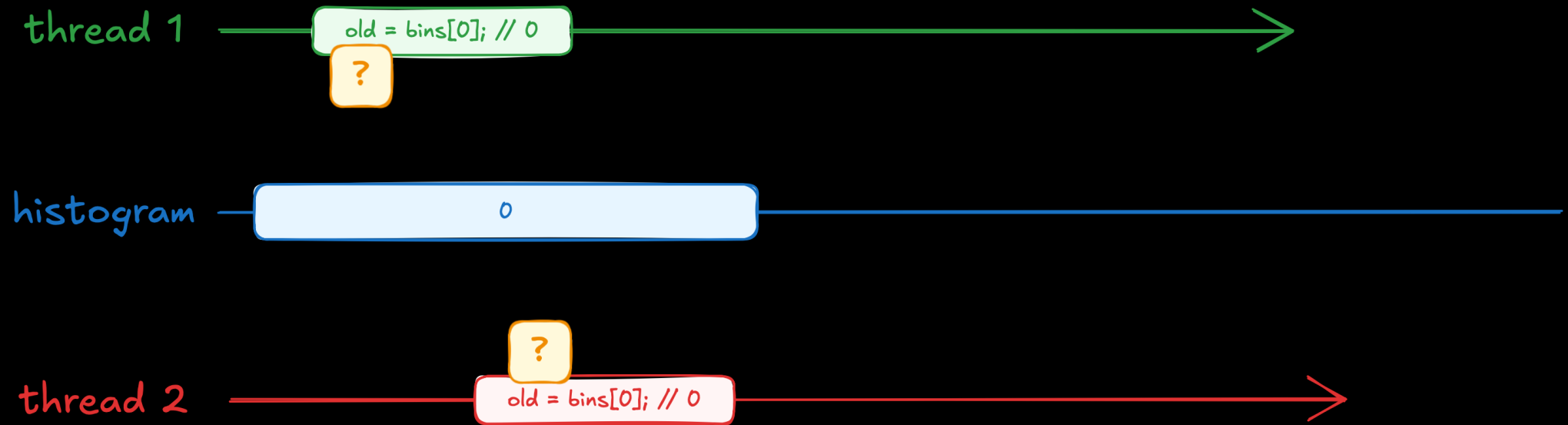
Top 20 Bins, Project Gutenberg Top Books, Size: 15MB

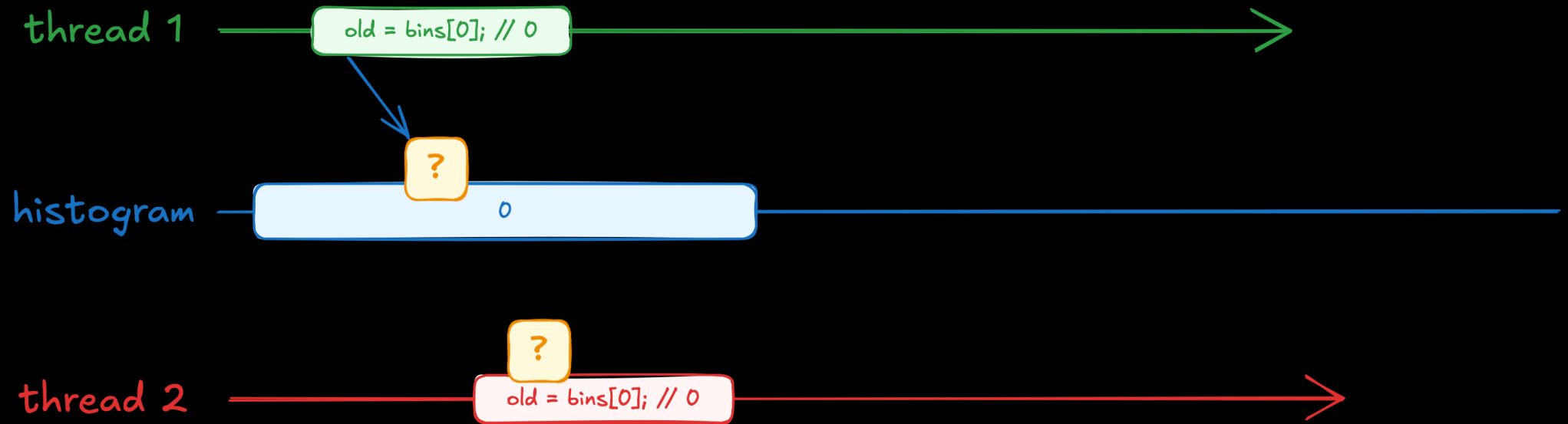


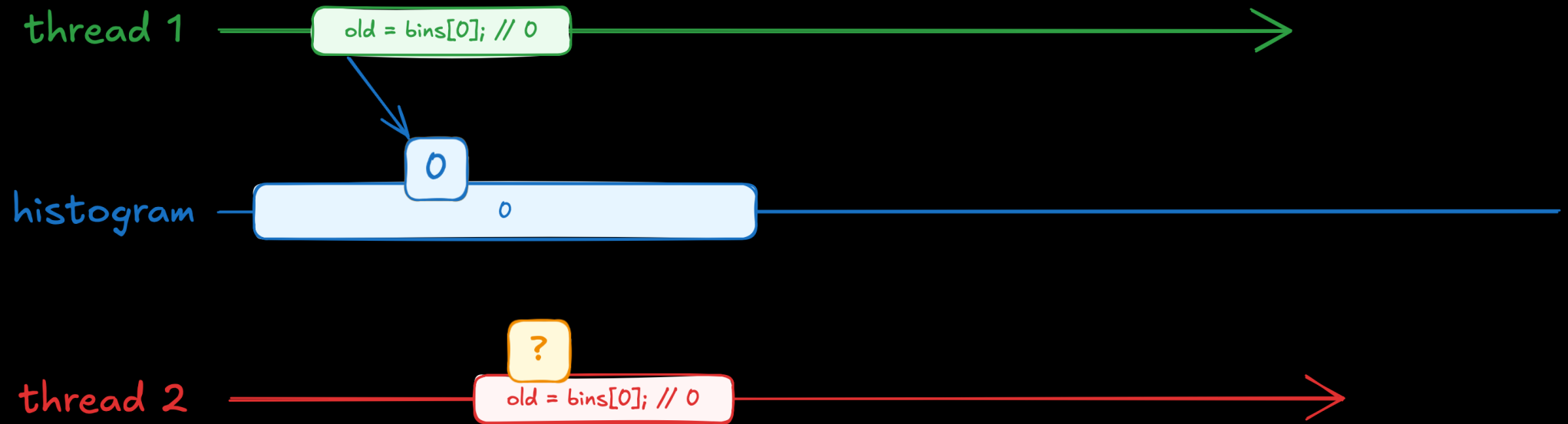
Top 20 Bins, Project Gutenberg Top Books, Size: 15MB

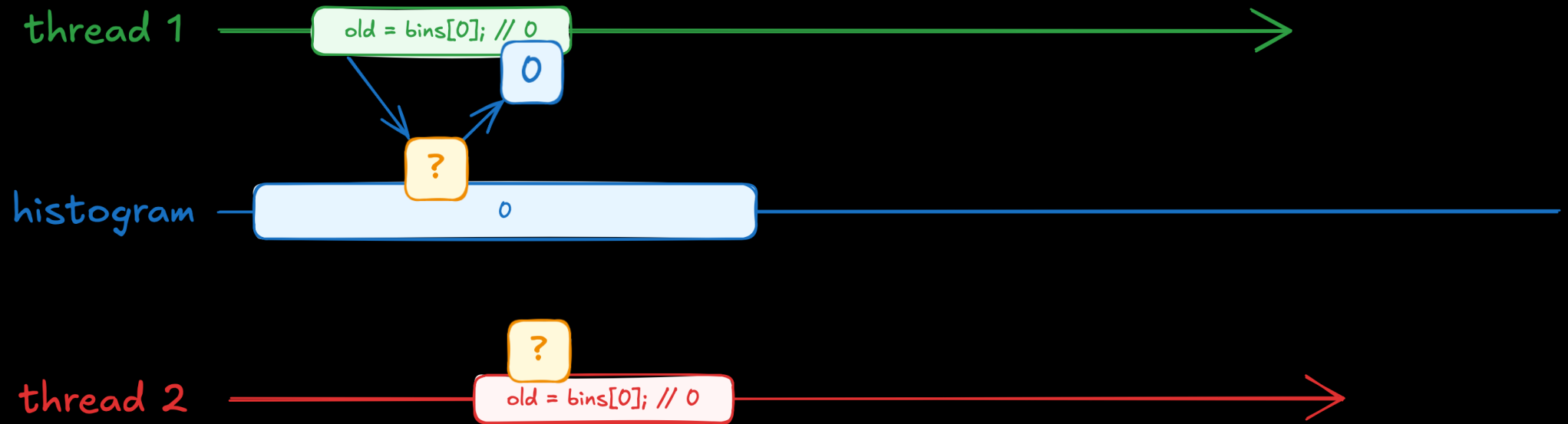


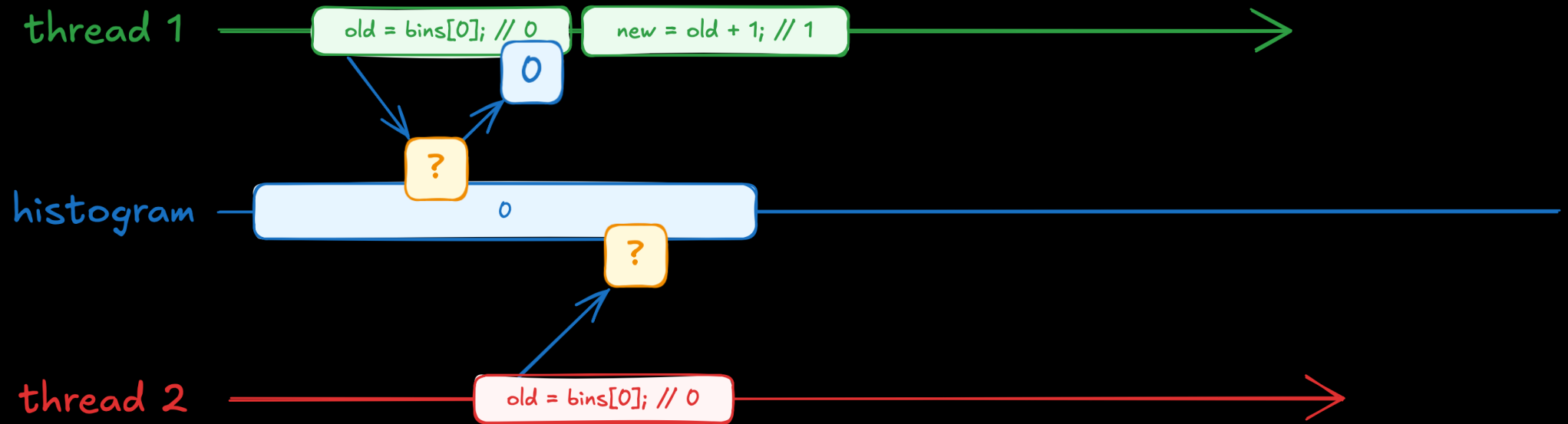
```
@cuda.jit
def histogram(values, bins, items):
    for i in range(items):
        value = values[cuda.grid(1) * items + i]
        old_count = bins[value]
        new_count = old_count + 1
        bins[value] = new_count
```

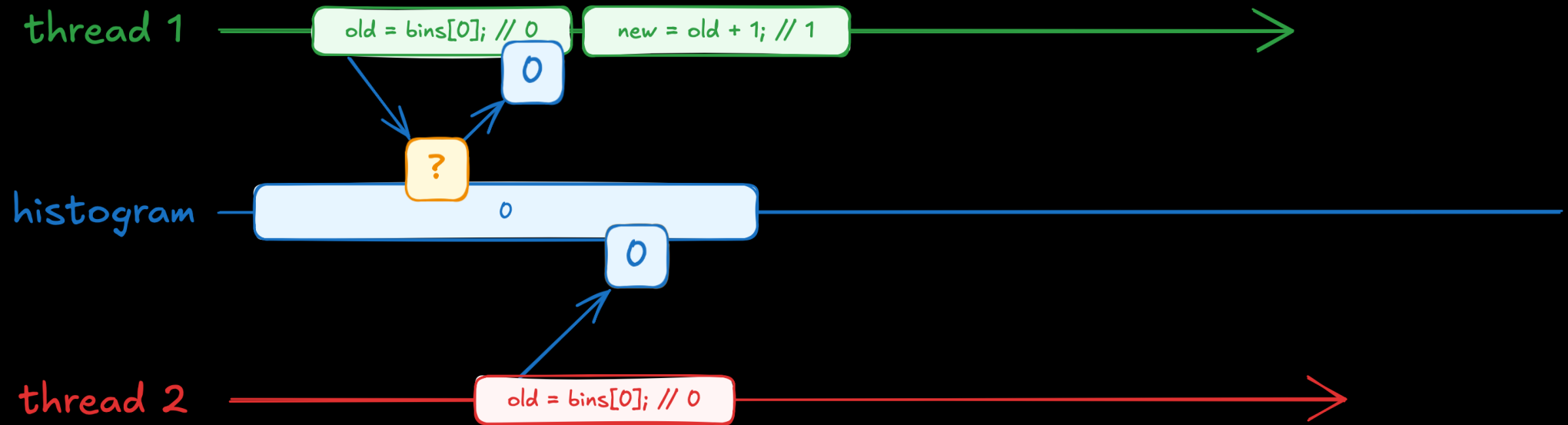



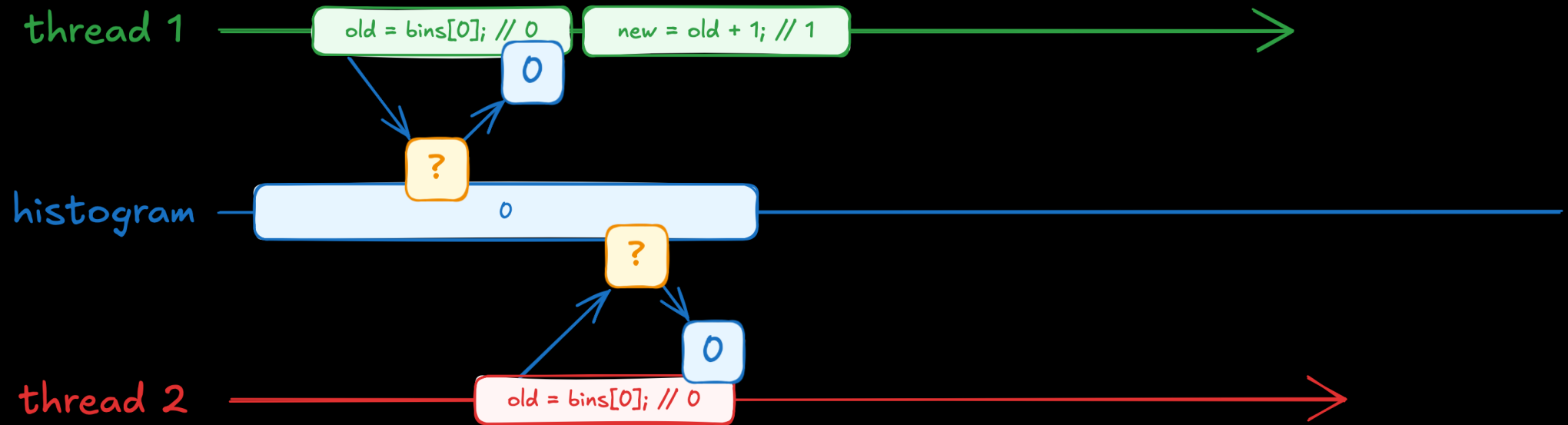


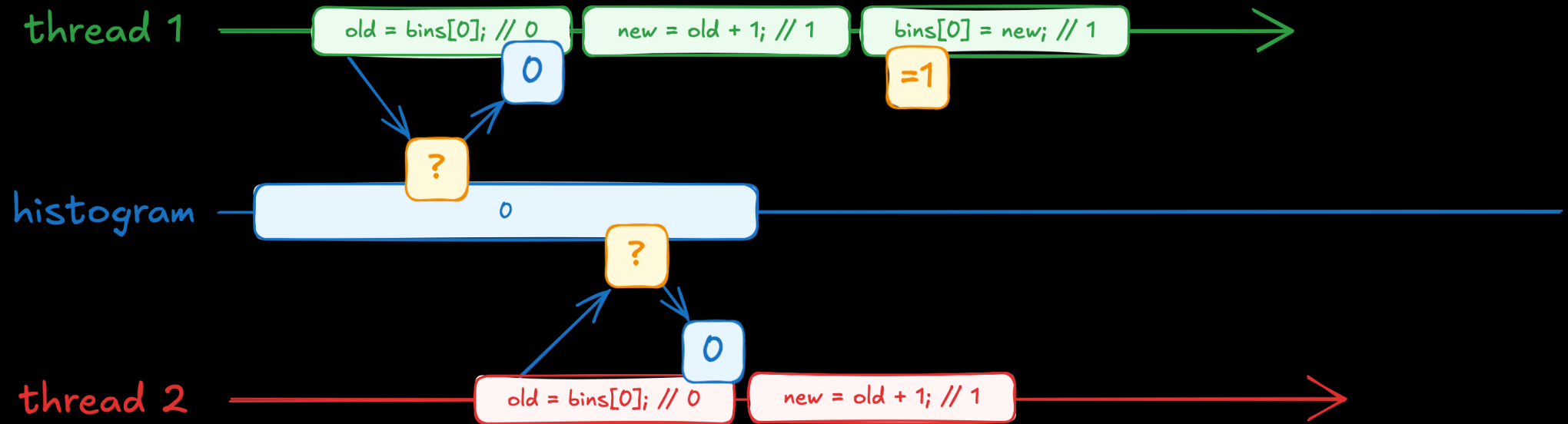


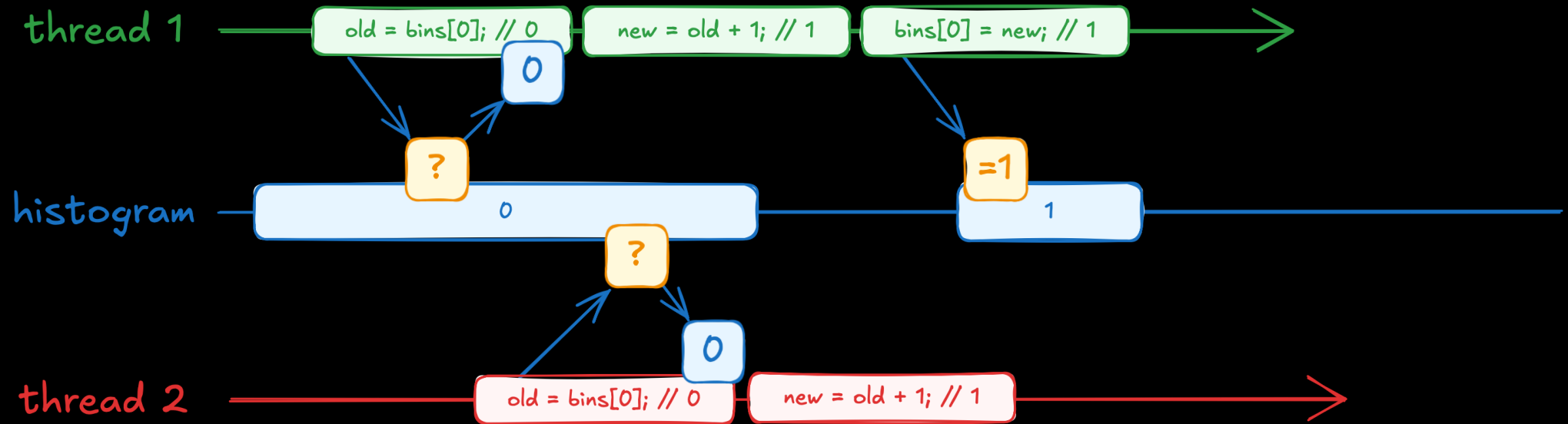


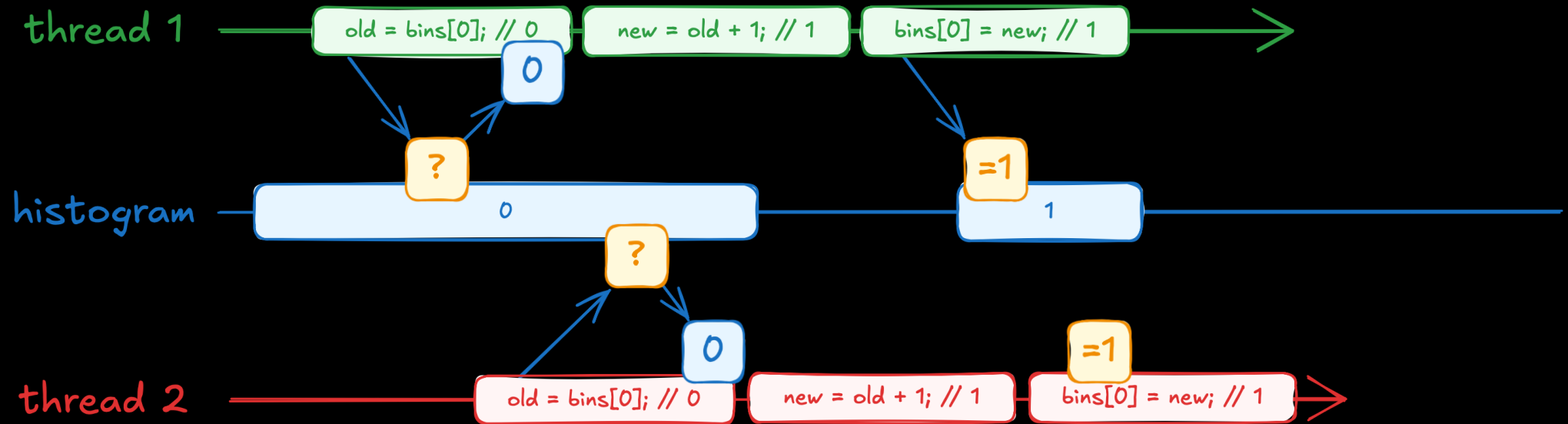


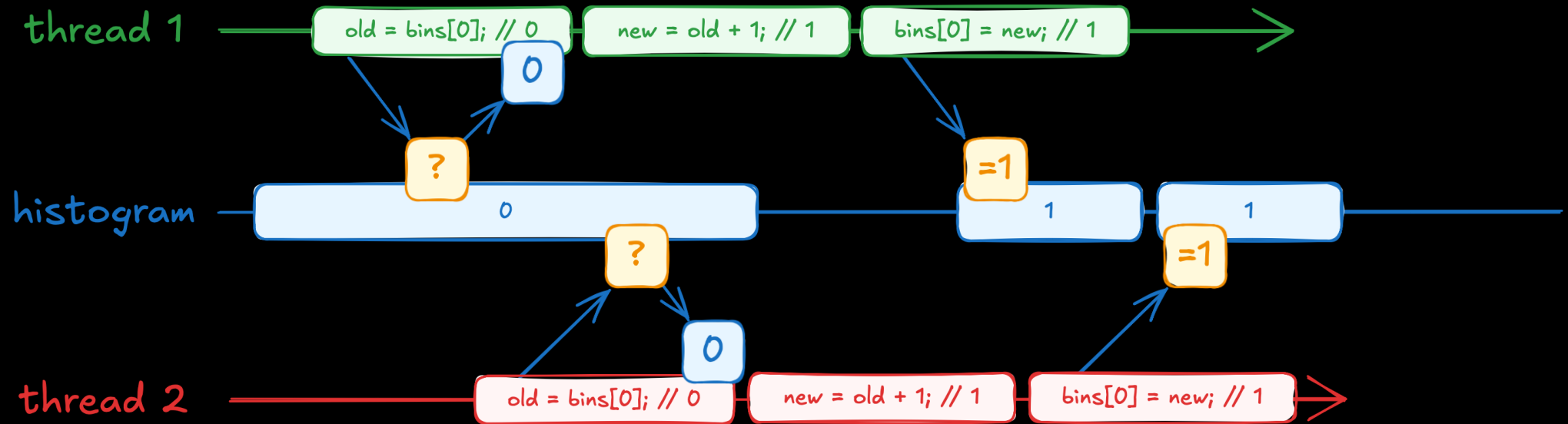








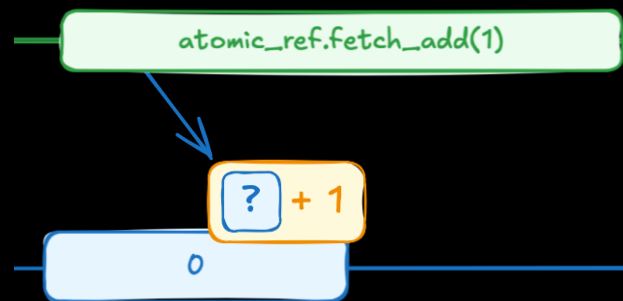
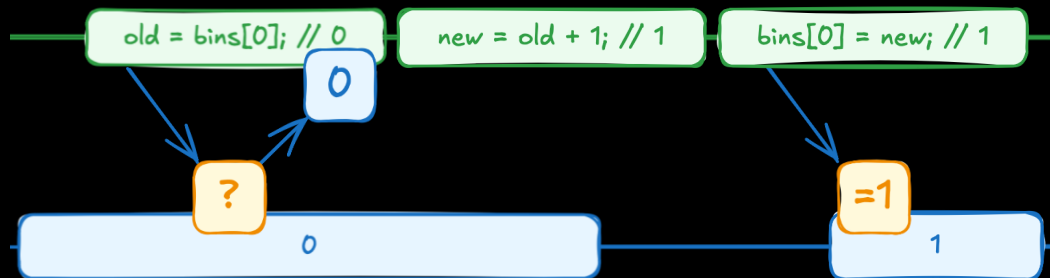




To fix the data race, the read, modify and write must be a single indivisible operation.

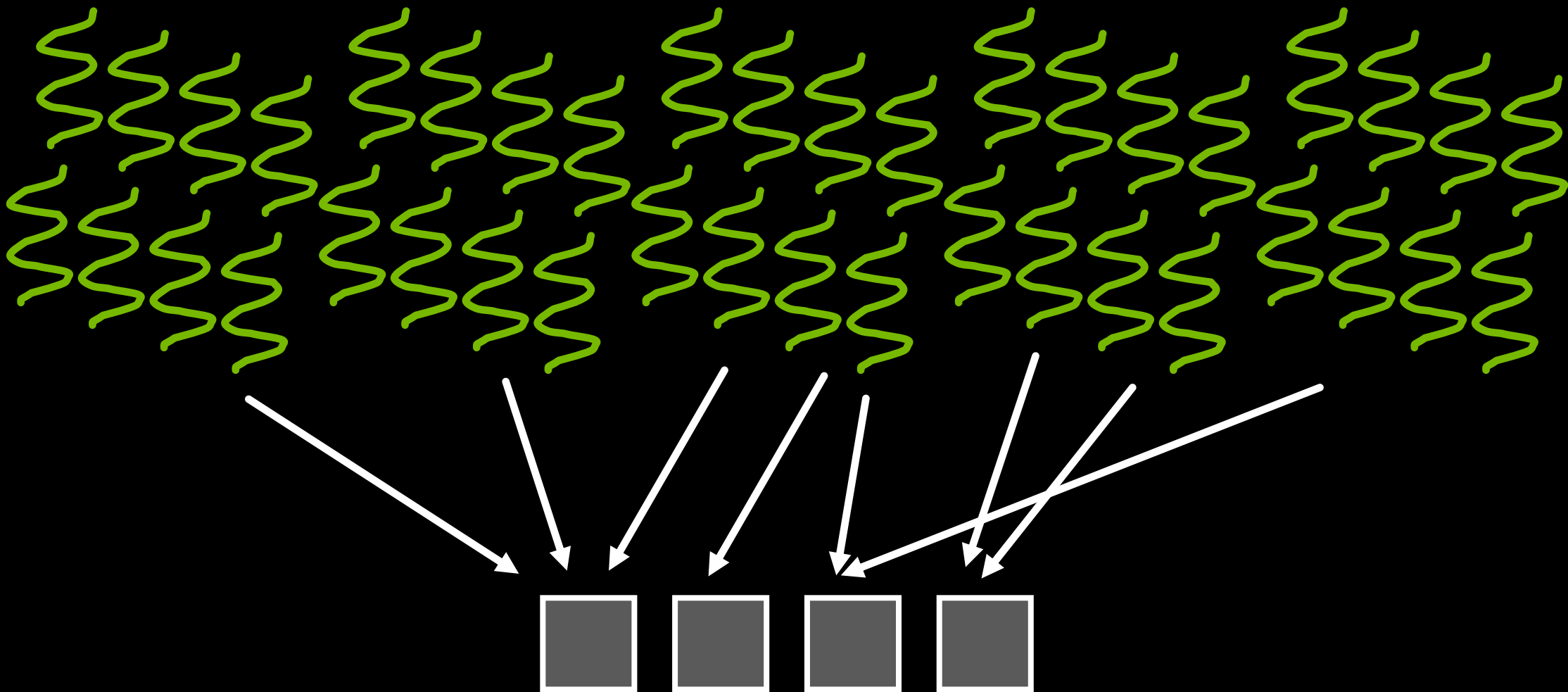
```
old_count = bins[value]
new_count = old_count + 1
bins[value] = new_count
```

```
cuda.atomic.add(
    bins, value, 1)
```



```
@cuda.jit
def histogram(values, bins, items):
    for i in range(items):
        value = values[cuda.grid(1) * items + i]
        old_count = bins[value]
        new_count = old_count + 1
        bins[value] = new_count
```

```
@cuda.jit
def histogram(values, bins, items):
    for i in range(items):
        value = values[cuda.grid(1) * items + i]
        cuda.atomic.add(bins, value, 1)
```



Block 0



Thread 0



Thread 1

Block 1

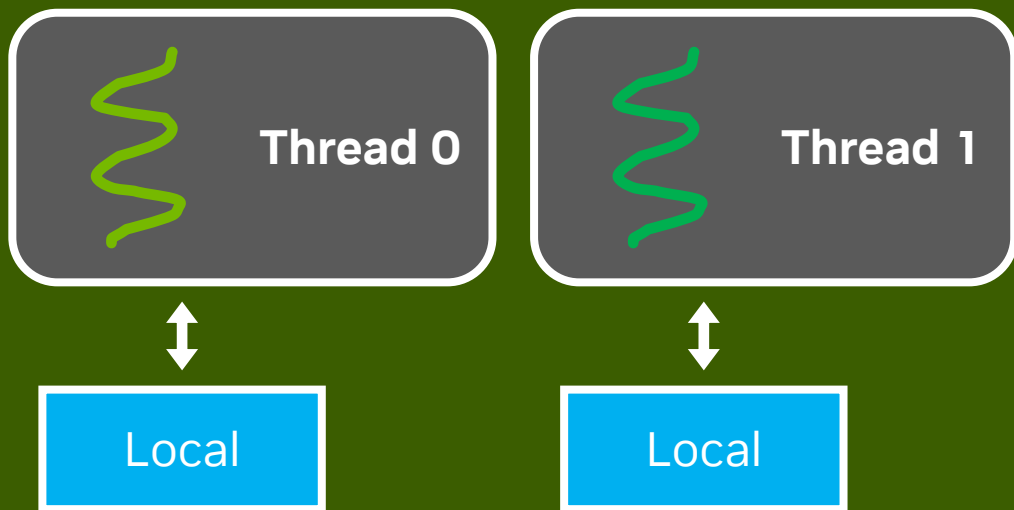


Thread 0

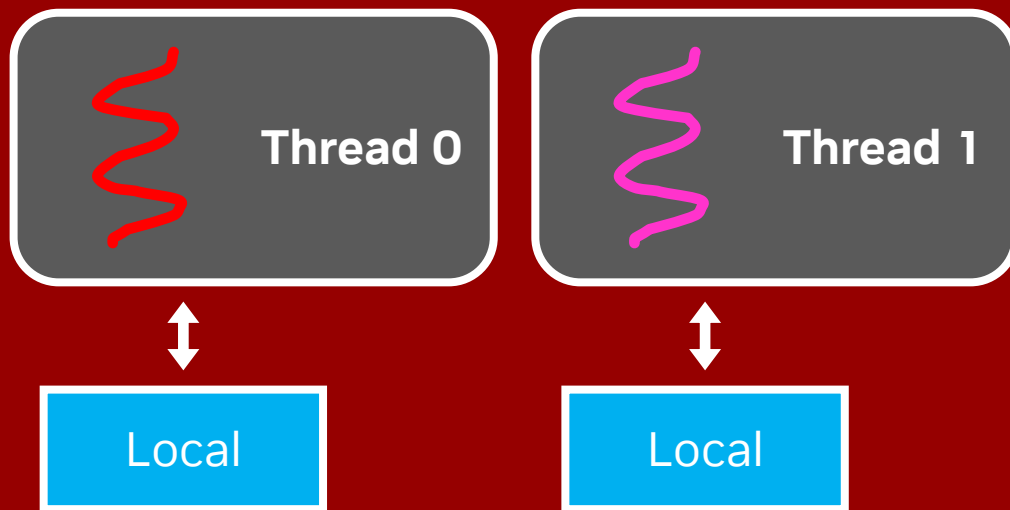


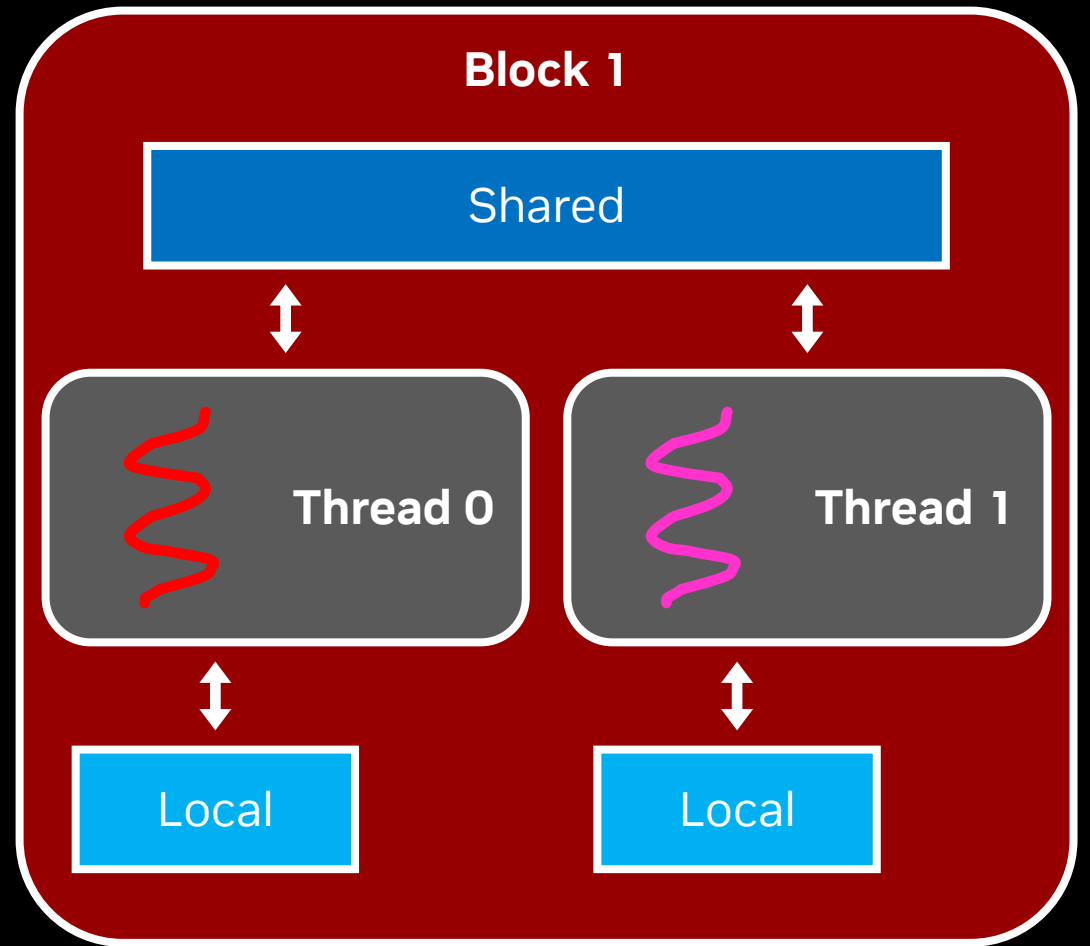
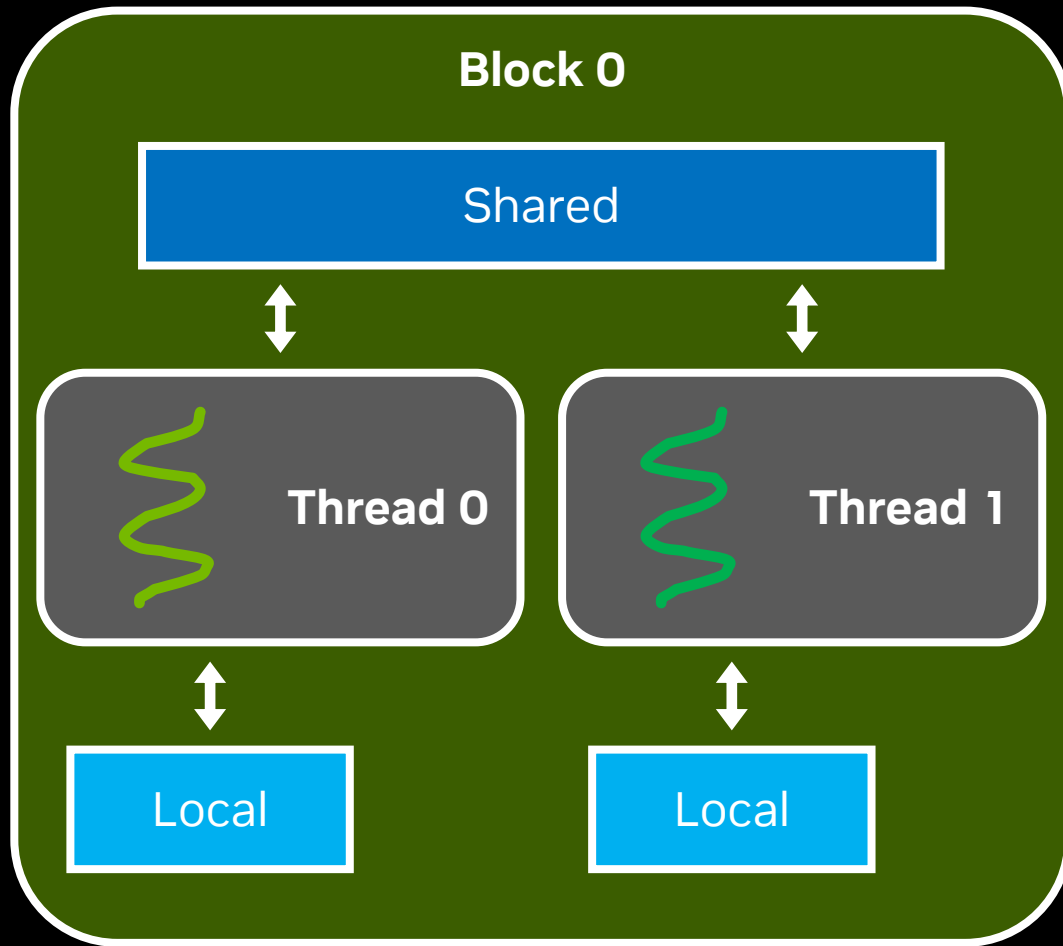
Thread 1

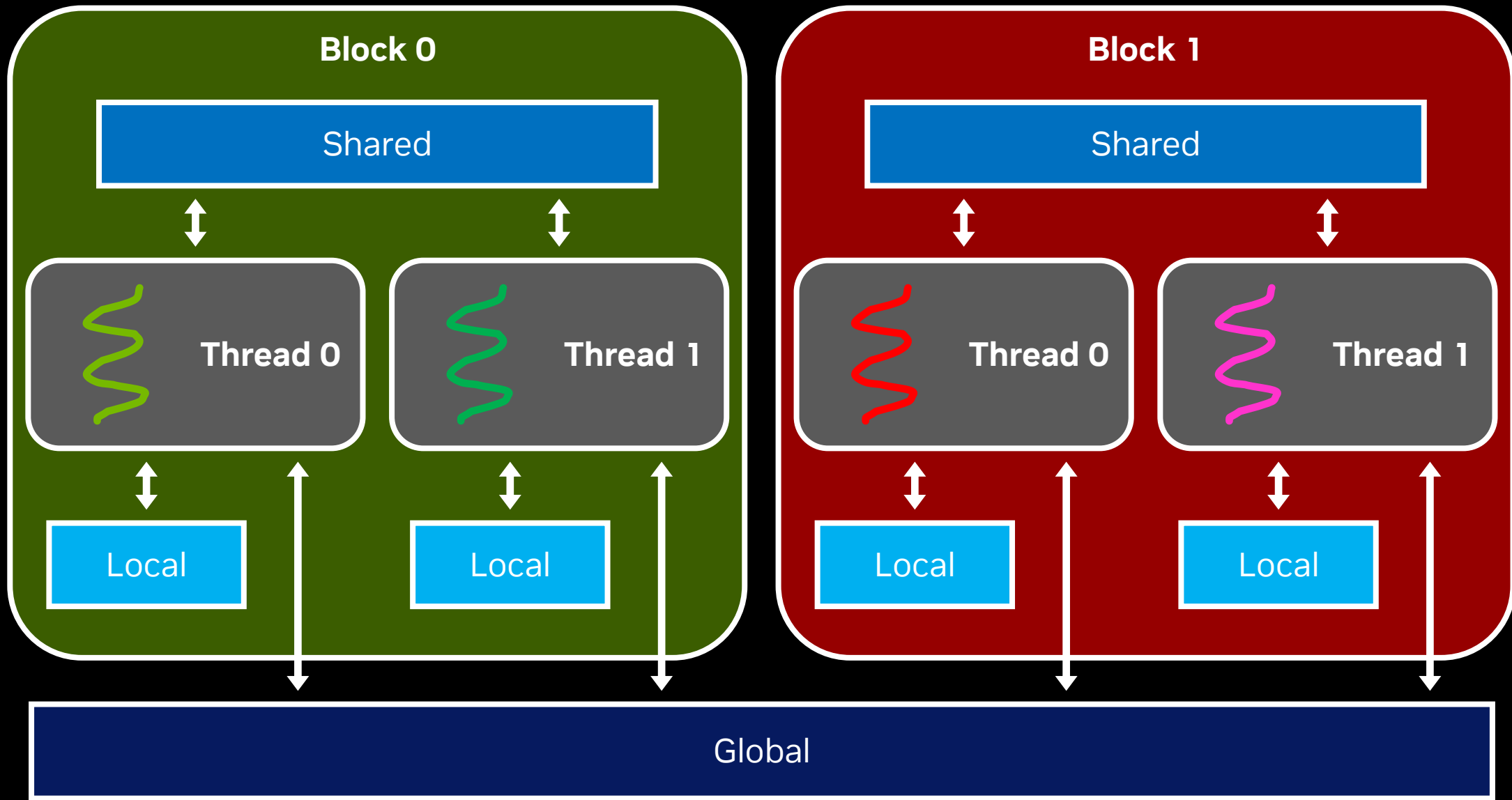
Block 0



Block 1







```
block_load = coop.block.load(cp.int32, 32, 1, 'striped')
block_store = coop.block.store(cp.int32, 32, 1, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def adjacent_difference(input, output):
    shared = cuda.shared.array(32, dtype=cp.int32)

    block_load(input, shared)

    if not threadIdx.x == 0:
        block_store(input,
            shared[threadIdx.x] - shared[threadIdx.x - 1])
```

```
block_load = coop.block.load(cp.int32, 32, 1, 'striped')
block_store = coop.block.store(cp.int32, 32, 1, 'striped')
```

```
@cuda.jit(link=block_load.files + block_store.files)
def adjacent_difference(input, output):
    shared = cuda.shared.array(32, dtype=cp.int32)

    block_load(input, shared)

    if not threadIdx.x == 0:
        block_store(input,
                    shared[threadIdx.x] - shared[threadIdx.x - 1])
```

```
block_load = coop.block.load(cp.int32, 32, 1, 'striped')
block_store = coop.block.store(cp.int32, 32, 1, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def adjacent_difference(input, output):
    shared = cuda.shared.array(32, dtype=cp.int32)

    block_load(input, shared)

    if not threadIdx.x == 0:
        block_store(input,
                    shared[threadIdx.x] - shared[threadIdx.x - 1])
```

```
block_load = coop.block.load(cp.int32, 32, 1, 'striped')
block_store = coop.block.store(cp.int32, 32, 1, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def adjacent_difference(input, output):
    shared = cuda.shared.array(32, dtype=cp.int32)

    block_load(input, shared)

    if not threadIdx.x == 0:
        block_store(input,
                     shared[threadIdx.x] - shared[threadIdx.x - 1])
```

```
block_load = coop.block.load(cp.int32, 32, 1, 'striped')
block_store = coop.block.store(cp.int32, 32, 1, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def adjacent_difference(input, output):
    shared = cuda.shared.array(32, dtype=cp.int32)

    block_load(input, shared)

    if not threadIdx.x == 0:
        block_store(input,
                    shared[threadIdx.x] - shared[threadIdx.x - 1])
```



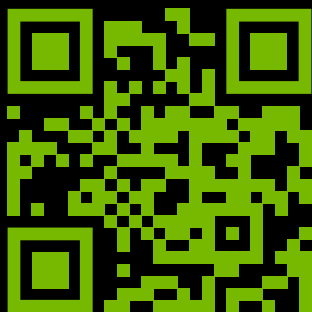
```
block_load = coop.block.load(cp.int32, 32, 1, 'striped')
block_store = coop.block.store(cp.int32, 32, 1, 'striped')

@cuda.jit(link=block_load.files + block_store.files)
def adjacent_difference(input, output):
    shared = cuda.shared.array(32, dtype=cp.int32)

    block_load(input, shared)

    cuda.syncthreads()

    if not threadIdx.x == 0:
        block_store(input,
                    shared[threadIdx.x] - shared[threadIdx.x - 1])
```



GTC CUDA Developer Track
nvda.ws/4dVjYXs



Accelerated Computing Hub
nvda.ws/43tdsU6

GENERAL CUDA

- [S72571](#) What's CUDA All About Anyways?
- [S72897](#) How To Write A CUDA Program
- [S72527](#) Debugging & Optimizing CUDA with Intelligent Developer Tools

CUDA PYTHON

- [S72450](#) Accelerated Python: Tour of Community & Ecosystem
- [S72448](#) The CUDA Python Developer's Toolbox
- [S72449](#) 1001 Ways to Write CUDA Kernels in Python
- [S74639](#) Tensor Core Programming in Python with CUTLASS 4.0

CUDA C++

- [S72574](#) Building CUDA Software at the Speed-of-Light
- [S72572](#) The CUDA C++ Developer's Toolbox
- [S72575](#) How You Should Write a CUDA C++ Kernel

MULTI-GPU PROGRAMMING

- [S72576](#) Getting Started with Multi-GPU Scaling: Distributed Libraries
- [S72579](#) Going Deeper with Multi-GPU Scaling: Task-based Runtimes
- [S72578](#) Advanced Multi-GPU Scaling: Communication Libraries

PERFORMANCE OPTIMIZATION

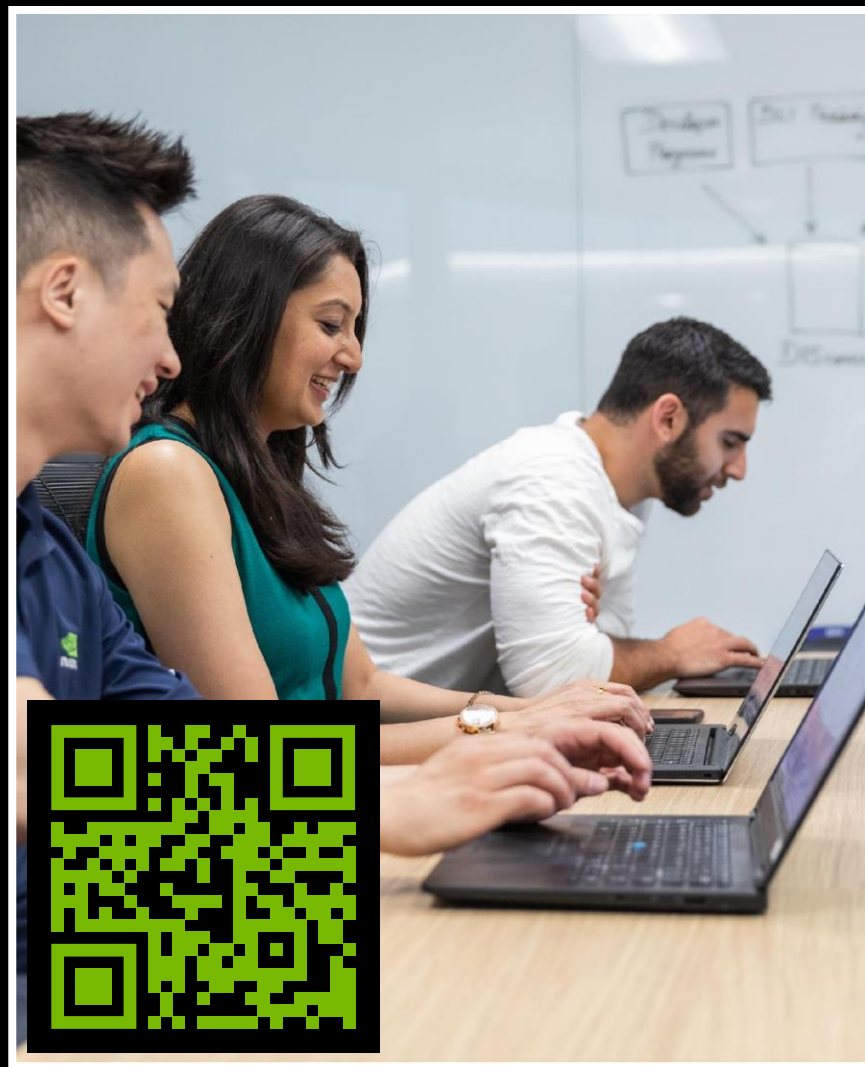
- [S72683](#) CUDA Techniques to Maximize Memory Bandwidth & Hide Latency
- [S72685](#) CUDA Techniques to Maximize Compute & Instruction Throughput
- [S72686](#) CUDA Techniques to Maximize Concurrency & System Utilization
- [S72687](#) Get the Most Performance From Grace Hopper

NVIDIA DLI Courses

- Fundamentals of Accelerated Computing with CUDA Python
- Fundamental of Accelerated Computing with Modern CUDA C++
- Fundamentals of Accelerated Data Science
- Accelerate Data Science Workflows with Zero Code Changes
- Optimizing CUDA Machine Learning Codes With Nsight Profiling Tools

Scan the QR code to access the full course list and redeem one free training!

nvidia.ws/3ZLPyAL



adspthepodcast.com



Algorithms +
Data
Structures =
Programs



```
constexpr std::array primes = {2, 3, 5, 7 // ... 97};

constexpr auto is_prime(int n) -> bool {
    return std::ranges::find(primes, n) != primes.end();
}

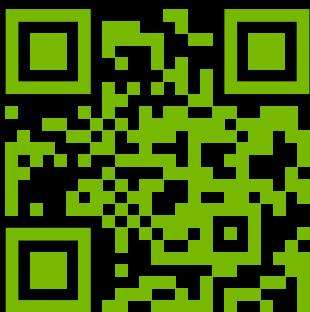
auto maximum_prime_difference(std::span<int const> nums) -> int {
    auto a = std::ranges::find_if(nums, is_prime);
    auto b = std::ranges::begin(std::ranges::find_last_if(nums, is_prime));
    return std::distance(a, b);
}
```



```
> [2,4,3] :: iota flat < prior : + * r . scan maxr add1
R      F < p δ + x r μ \ m '
[2,4,3] [1, 2], [1, 2, 3, 4], [1, 2, 3]
RF      [2,4,3] [1, 2, 1, 2, 3, 4, 1, 2, 3]
RF<     [2,4,3] [1, 1, 1, 2, 3, 4, 1, 2, 3]
RF<p    [2,4,3] [1, 0, 1, 1, 1, 0, 1, 1]
RF<pδ+  [2,4,3] [3, 4, 4, 1, 1, 0, 1, 1]
RF<pδ+x [2,4,3] [3, 0, 4, 2, 2, 0, 2, 2]
RF<pδ+xr [2,4,3] [6, 16, 12, 1, 1, 0, 1, 1]
RF<pδ+xrμ [2,4,3] [1, 0, 1, 2, 3, 0, 1, 2]
RF<pδ+xrμm [2,4,3] 3
RF<pδ+xrμm' [2,4,3] 4

This is a 1-1-2-q-s-2-2-2-s-q-1-1 monadic chain
      B h1 Φ1
      B h1
      B h1
      B
      B
      B
```





**GTC CUDA
Developer Track**
nvidia.ws/4dVjYXs



**Accelerated
Computing Hub**
nvidia.ws/43tdsU6



**NVIDIA
DLI Courses**
nvidia.ws/3ZLPyAL



**ADSP
Podcast**
adspthepodcast.com

KERNEL AUTHORING WITH CUDA PYTHON